# Combining data: merge, append, lag

## Programming & Data Management, Part II (Python) · Session 3

Magnus Lodefalk
*Örebro University*

NA446A, Örebro University

Autumn 2026



# Today's goals

By the end of today you can:

1. Merge two tables, as with Stata's `merge`, and check that no row vanished or multiplied
2. Append yearly files, as with Stata's `append`
3. Make a lag, as with Stata's `L.`.

# From last time

Answer on your own first, then compare with a neighbour

1. A person's age is missing. What do Stata's `gen old = age > 65` and pandas' `(df["age"] > 65).astype(int)` give that person?
2. Each worker's firm mean wage on the worker's own row: `agg` or `transform`?

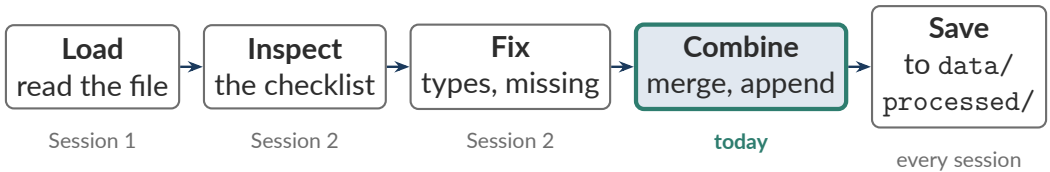
# From last time

Answer on your own first, then compare with a neighbour

1. A person's age is missing. What do Stata's `gen old = age > 65` and pandas' `(df["age"] > 65).astype(int)` give that person?
   Answer: Stata 1 (missing counts as larger than any number); pandas 0 (`NaN > 65` is False). Both are wrong: keep it missing
2. Each worker's firm mean wage on the worker's own row: `agg` or `transform`?
   Answer: `transform`: one value per row. Today a merge does the same when the firm values sit in another table

# Every data task follows one recipe

Today: combine




- You have done all of today's tasks in Stata. Today is new words for old ideas.

# Today in Stata and in pandas

| Task | Stata | pandas |
|---|---|---|
| is the id unique? | `isid lopnr` | `persons["LopNr"].is_unique` |
| add columns by id | `merge m:1 firm using firms` | `people.merge(firms, on="firm")` |
| what matched? | `_merge` (1, 2, 3) | `indicator=True` |
| promise the key shape | the `m:1` in `merge` | `validate="m:1"` |
| drop exact copies | `duplicates drop` | `drop_duplicates()` |
| stack yearly files | `append using` | `pd.concat(frames)` |
| last year's value | `L.inc` after `xtset` | `groupby(...)["inc"].shift(1)` |

- Same seven jobs, new words. Every code slide today shows the Stata first.

# How to read one line of pandas

```
m = people.merge(firms, on="firm", how="left")
```

| Part | What it means | In Stata |
|---|---|---|
| `people` | the left table | the data in memory (master) |
| `.merge(...)` | what to do with it (a *method*) | the command `merge` |
| `firms` | the right table | `using firms` |
| `on="firm"` | the key: match on this column | `m:1 firm` |
| `how="left"` | which unmatched rows to keep | `keep(master match)` |
| `m =` | save the result under the name `m` | (Stata replaces the data in memory) |
| `m` on its own line | show the table (last line of a cell) | `list` |

- Stata has one dataset in memory. pandas has many tables, each with a name, and nothing changes until you write `name =`.

# Merge

# Steps 1–5: count the rows in and out

| | Question | Stata you know |
|---|---|---|
| 1 | what does a merge keep? | `merge` keeps all, `_merge` says which |
| 2 | which rows matched? | `tab _merge` |
| 3 | how do we stop the rows from multiplying? | `merge m:1` stops by itself |
| 4 | which ids repeat? | `duplicates report` |
| 5 | did every row match, and none multiply? | `merge, assert(master match)` |

- After every merge, ask: how many rows went in, and how many came out?

# Now I'll show you in VS Code

Cells K1, M0, M1, M2: the key, the two tables, Steps 1–2

# First check the key: is each id there only once?

**Stata:** `isid lopnr` → "variable lopnr does not uniquely identify the observations"

```
RAW = Path("pair_00/data/raw")
persons = pd.read_csv(RAW / "persons.csv", dtype=str)
print(len(persons), persons["LopNr"].nunique())
print(persons["LopNr"].is_unique)
```

```
1005 1000
False
```

- 1,005 rows, 1,000 persons: some ids repeat. A merge on `LopNr` would inherit the problem.

`len`: number of rows (`count`); `nunique()`: number of different ids; `is_unique`: True if none repeats.

# Two small tables to see what a merge does

```
people = pd.DataFrame({"pid": [1, 2, 3, 4],
                       "firm": ["A", "B", "B", "C"]})
firms = pd.DataFrame({"firm": ["A", "B", "D"],
                      "size": [12, 40, 7]})
```

| | pid | firm |
|---|---|---|
| 0 | 1 | A |
| 1 | 2 | B |
| 2 | 3 | B |
| 3 | 4 | C |

| | firm | size |
|---|---|---|
| 0 | A | 12 |
| 1 | B | 40 |
| 2 | D | 7 |

Firm C has no row in `firms`; firm D has no people.

`pd.DataFrame({column: values})` types a small table by hand, like Stata's `input`.

- Many people, one row per firm: in Stata words, an `m:1` merge.

# Step 1: what does a merge keep?

**Stata:** `merge m:1 firm using firms` → all 5 rows stay: 3 matched, 1 master only, 1 using only

```
m = people.merge(firms, on="firm")
m   # show the table (Stata: list)
```

```
   pid firm  size
0    1    A    12
1    2    B    40
2    3    B    40
```

pandas: three rows. Person 4 (firm C) and firm D are gone.

- The opposite default: Stata keeps everything, pandas keeps only the ids found in *both* tables, and says nothing.

# Step 2: which rows matched? (pandas' `_merge`)

```
m = people.merge(firms, on="firm",
                 how="outer", indicator=True)
m   # show the table (Stata: list)
```

| | pid | firm | size | _merge |
|---|---|---|---|---|
| 0 | 1.0 | A | 12.0 | both |
| 1 | 2.0 | B | 40.0 | both |
| 2 | 3.0 | B | 40.0 | both |
| 3 | 4.0 | C | NaN | left_only |
| 4 | NaN | D | 7.0 | right_only |

| Stata | pandas |
|---|---|
| 1 master only | `left_only` |
| 2 using only | `right_only` |
| 3 matched | `both` |

`how="outer"`: keep all rows, as Stata does. `indicator=True`: add `_merge`.

- With these two arguments, pandas behaves like Stata's `merge`.

`pid` 1.0: a column that gets a missing value turns decimal.

# Firm B twice: how many rows after a left merge?

Your bet first: hands up. The answer is on the next slide.

`people`

| pid | firm |
|---|---|
| 1 | A |
| 2 | B |
| 3 | B |
| 4 | C |

`firms2`

| firm | size |
|---|---|
| A | 12 |
| B | 40 |
| B | 45 |

Left merge on `firm`: keep every person.

A. 4: one row per person

B. 5

C. 6

# A left merge kept every person and still added rows

```
m = people.merge(firms2, on="firm", how="left")
m   # show the table (Stata: list)
```

| | pid | firm | size |
|---|---|---|---|
| 0 | 1 | A | 12.0 |
| 1 | 2 | B | 40.0 |
| 2 | 2 | B | 45.0 |
| 3 | 3 | B | 40.0 |
| 4 | 3 | B | 45.0 |
| 5 | 4 | C | NaN |

Persons 2 and 3 match both B rows, so each appears twice.

Step 2 had 5 rows because the outer merge also kept firm D. Here the left merge keeps only people, and there is no D.

- 4 people in, 6 rows out. Nothing warns you: the key on the right was not unique.

Stata would have refused this merge. Next slide: how to make pandas refuse too.

# Step 3: make pandas refuse, as Stata does

**Stata:** `merge m:1 firm using firms2` → "variable firm does not uniquely identify observations in the using data"

```
people.merge(firms2, on="firm", how="left",
             validate="m:1")
```

```
MergeError: Merge keys are not unique in right
dataset; not a many-to-one merge
```

- Stata makes you write `m:1`; pandas does not. So always write `validate="m:1"`: it is the same promise, and pandas stops when the data break it.

▷ 1:1, m:1, 1:m, m:m ▷ The same on register data

# Step 4: which ids repeat, and are they copies?

**Stata:** `duplicates report lopnr` · `duplicates report` · `duplicates drop`

```
# does any person appear more than once?
print(persons.duplicated("LopNr").sum())
# is that because the same row was recorded twice?
print(persons.duplicated().sum())
persons = persons.drop_duplicates()
```

```
5
3
```

- 5 repeated ids: 3 are exact copies (drop them), 2 are persons with two *different* records. For those, decide, and write down what you decided.

`duplicated()` marks each row that repeats an earlier one; `.sum()` counts them (True counts as 1).

# Step 5: did every row match, and none multiply?

**Stata:** `merge m:1 lopnr using persons, assert(master match)`

```
lisa23 = pd.read_csv(RAW / "lisa_2023.csv", dtype=str)
persons = persons.drop_duplicates("LopNr")
m = lisa23.merge(persons, on="LopNr", how="left",
                 validate="m:1", indicator=True)
assert len(m) == len(lisa23)
m["_merge"].value_counts().to_dict()  # count each value
```

```
{'both': 971, 'left_only': 0, 'right_only': 0}
```

- Four parts, every time: `how`, `validate`, `indicator`, `assert`. All 971 rows match; none added.

`drop_duplicates("LopNr")`: first record of each id; `assert`: stop if not true; `value_counts()`: `tab`. All side by side: appendix A1.

# Append and lag

# Steps 6–9: stack the years, then look back one year

| | Question | Stata you know |
|---|---|---|
| 6 | how do we stack five yearly files? | `foreach` and `append using` |
| 7 | a variable changed name: what now? | `rename`, then `append` |
| 8 | are person and year a key? | `isid lopnr year` |
| 9 | what was each person's income last year? | `xtset`, then `L.inc` |

- First one new piece of Python: the loop.

# Now I'll show you in VS Code

Cells L0 and A1: the loop, Step 6

# New Python: a loop, as Stata's `foreach`

**Stata**

```
foreach y of numlist 2019/2023 {
    display "lisa_`y'.csv"
}
```

**pandas**

```
for y in range(2019, 2024):
    print(f"lisa_{y}.csv")
```

```
lisa_2019.csv
lisa_2020.csv
lisa_2021.csv
lisa_2022.csv
lisa_2023.csv
```

`range(2019, 2024)` 2019 to 2023: Python stops *before* the last number

`f"lisa_{y}.csv"` text with the value of `y` put in, like Stata's `` `y' ``

the colon and the indent what Stata's `{ }` do: the indented lines repeat

# Step 6: how do we stack five yearly files?

```
frames = []
for y in range(2019, 2024):
    d = pd.read_csv(RAW / f"lisa_{y}.csv", dtype=str)
    frames.append(d.assign(year=y))
pd.concat(frames).shape
```

```
(4884, 27)
```

`frames = []`: an empty list, a box to collect the five tables in. `frames.append(...)` puts one table in the box: it is *not* Stata's `append`. `assign(year=y)`: `gen year = `y'`. `pd.concat` stacks the box: that is Stata's `append`.

- 4,884 rows: 975 + 984 + 978 + 976 + 971. The rows add up. The columns do not: next question.

# Check your understanding

Answer on your own first, then compare with a neighbour

Each yearly file has 21 columns, with `year`. The stack has 27. Where do the extra columns come from? (`concat`, like Stata's `append`, lines columns up by name.)

# Check your understanding

Answer on your own first, then compare with a neighbour

Each yearly file has 21 columns, with `year`. The stack has 27. Where do the extra columns come from? (`concat`, like Stata's `append`, lines columns up by name.)

**Answer:** From variables that were *renamed* in 2021. The old name is empty from 2021 and the new name before 2021, so one variable sits in two half-empty columns. Stata's `append` does exactly the same.

# Step 7: rename before you stack

**Stata:** `capture rename syssstat19 syssstat11` (inside the loop, before `append`)

```
frames = []
for y in range(2019, 2024):
    d = pd.read_csv(RAW / f"lisa_{y}.csv", dtype=str)
    d = d.rename(columns={"SyssStat19": "SyssStat11"})
    frames.append(d.assign(year=y))
lisa = pd.concat(frames, ignore_index=True)
lisa.shape
```

```
(4884, 26)
```

- One column fewer. In the lab: does the codebook list every rename?

`rename` takes {old: new}. `ignore_index=True`: number the rows 0, 1, 2, … again.

# Step 8: are person and year a key?

**Stata:** `duplicates report lopnr year · duplicates drop · isid lopnr year`

```
print(lisa.duplicated(["LopNr", "year"]).sum())
print(lisa.duplicated().sum())
lisa = lisa.drop_duplicates()
print(lisa.duplicated(["LopNr", "year"]).sum(),
      len(lisa))
```

```
1
1
0 4883
```

- Step 4 again, with two key columns: one exact copy, dropped. Now person and year identify each row: 4,883 person-years.

# Step 9: what was each person's income last year?

**Stata:** `xtset lopnr year · gen inc_lag = L.inc`

```
lisa["inc"] = pd.to_numeric(lisa["ForvErs"])
lisa = lisa.sort_values(["LopNr", "year"])
lisa["inc_lag"] = lisa.groupby("LopNr")["inc"].shift(1)
lisa[["LopNr", "year", "inc", "inc_lag"]].head(3)
```

| | LopNr | year | inc | inc_lag |
|---|---|---|---|---|
| 0 | P000001 | 2019 | 2724 | NaN |
| 975 | P000001 | 2020 | 967 | 2724.0 |
| 1959 | P000001 | 2021 | 1029 | 967.0 |

- `shift(1)`: the row above, within each person, like `L.`, except across a missing year (appendix A4).

# Today's lab

1. Unzip `s3_lab.zip` into `na446a-starter`; open `s3_lab/s3_lab.py`. Everyone uses pair 00: it is already in the folder
2. In class: Steps 1–4. Each step shows the Stata line you would write; you write the pandas. Check with `check_s3.py`
3. Step 4: catch the wrong merge an AI proposes: predict its rows, run it, name two faults, write the right merge
4. At home: Steps 5–7, then the exercise (the bug hunt)

**Now I'll show you in VS Code**

- A merge that runs is not always right: `validate`, `indicator`, `assert`.

The data are synthetic; never paste real personal or register data into a chatbot.

# Summary

1. Merge: check the key first. pandas keeps only matches unless you say `how=`; write `validate="m:1"` as you write `m:1` in Stata; `indicator=True` is `_merge`; `assert` the rows
2. Append: a loop collects the yearly tables, `pd.concat` stacks them; rename *before* you stack, then check the new key (person and year)
3. Lag: sort, `groupby`, `shift(1)`, as `L.` in Stata, except across a missing year

**Before next time:** lab Steps 5–7, then the bug hunt (course site), with its self-check; about 2 hours. Self-study: the appendix slides on reshape and the lag across a gap. Before Session 4: Gentzkow and Shapiro, chapter 2, *Automation*.

# Test yourself before next time

Without AI: this is what you should manage on your own

1. What does pandas' `merge` keep by default, and Stata's? Write the pandas line that keeps what Stata keeps and shows what matched.
2. 1,000 workers, left-merged onto a firm file in which one firm with 30 workers appears twice. How many rows? Which argument would have stopped it?
3. You append five yearly files and get two income columns, each half empty. What happened, and where in the code is the fix?
4. Wages 100 in 2021, 120 in 2023, no row for 2022: the 2023 lag with `shift(1)` by person, and with Stata's `L.wage`?
5. (Self-study, appendix) A table has 290 municipalities and one column per year, 2015–2024. Write the line that makes it long. How many rows?

# Appendix

# pandas and Stata side by side

| Task | pandas | Stata |
|---|---|---|
| Is it a key? | `df["id"].is_unique` | `isid id` |
| Merge, keep all | `how="outer", indicator=True` | `merge, _merge` 1/2/3 |
| Keep the left table | `how="left"` | `keep(master match)` |
| Keep matches only | `how="inner"` (the default) | `keep(match)` |
| Promise the key shape | `validate="1:1", "m:1", "1:m"` | `merge 1:1, m:1, 1:m` |
| Check the result | `assert len(m) == len(left)` | `assert(master match)` |
| Stack rows | `pd.concat([a, b])` | `append using` |
| Wide to long | `melt` | `reshape long` |
| Long to wide | `pivot` | `reshape wide` |
| Lag | `groupby(id)[x].shift(1)` (a row) | `L.x` after `xtset` (a period) |

# 1:1, m:1, 1:m, m:m: which merge is which?

- **1:1**: one row per key on both sides: 2023 person file and 2023 grades per person
- **m:1**: many rows on the left, one on the right: person-years and the person file; workers and firms
- **1:m**: the same, the other way round: firms and their workers
- **m:m**: many on both sides. Every left row meets every right row with the same key, so the rows multiply. Almost always a sign that one table should first be aggregated to the key of the other

- Name the shape before you merge, and write it into `validate=`.

Stata's `merge m:m` does not form all pairs, and its own manual advises against it; `joinby` does. Either way: aggregate first.

◁ back

# The same failure on the register data

- `lisa23` is the 2023 person file (971 rows); `persons`, as loaded, repeats 5 ids
- `lisa23.merge(persons, on="LopNr", how="left", validate="m:1")` stops with the same `MergeError`: `LopNr` is not unique in the *right* table
- Without `validate` it runs and returns 976 rows for 971 persons, with no warning

- The toy example's fault, on real-sized data: Step 4 finds the duplicates.

Stata: `merge m:1 lopnr using persons` refuses, r(459).

# Self-study: the lag across a missing year

```
prev = lisa.groupby("LopNr")["year"].shift(1)
lisa["gap"] = lisa["year"] - prev
print(lisa["gap"].value_counts().to_dict())
lisa["inc_lag"] = lisa["inc_lag"].where(lisa["gap"] == 1)
print(lisa["inc_lag"].notna().sum())
```

```
{1.0: 3831, 2.0: 51, 3.0: 1}
3831
```

- 52 “lags” jump over a year in which the person is missing. Keep a lag only where the gap is one year: 3,831 remain.

Persons leave a yearly file for a year (abroad, say). Stata’s `L.` uses the calendar after `xtset`, so it gives missing across a gap. `where(condition)` keeps the value where the condition holds, else missing (Session 2).

# Self-study: long to wide (Stata's `reshape wide`)

```
emp = lisa[["LopNr", "year", "SyssStat11"]]
wide = emp.pivot(index="LopNr", columns="year",
                 values="SyssStat11")
wide.head(3)
```

| year | 2019 | 2020 | 2021 | 2022 | 2023 |
|---|---|---|---|---|---|
| LopNr | | | | | |
| P000001 | 1 | 6 | 5 | 5 | 6 |
| P000002 | 1 | 1 | 1 | 1 | 1 |
| P000003 | 1 | 5 | 1 | 1 | NaN |

- One row per person, one column per year, as in a table in a paper. `NaN`: P000003 is not in the 2023 file.

`index`: Stata's `i()`; `columns`: `j()`; `values`: the variable. Codes: 1 employed, 5 and 6 not.

# Self-study: wide to long (Stata's `reshape long`)

```
long = wide.reset_index().melt(id_vars="LopNr",
                               value_name="status")
print(len(long))
long = long.dropna()
print(len(long), list(long.columns))
```

```
5000
4883 ['LopNr', 'year', 'status']
```

- 1,000 persons $\times$ 5 years = 5,000 cells. 117 were empty because that person-year does not exist; after `dropna` we are back at 4,883.

`reset_index`: `LopNr` back to a column; `melt`: the year columns become rows; `id_vars`: Stata's `i()`. Keep panels long while you work, wide for the final table. Stata's `reshape long` also keeps the empty cells.

# Self-study: the codebook is data too, and it can be wrong

The register lab's codebook disagrees with its data in three places

- It was written by people, from an earlier delivery, as real codebooks are
- The lab's Step 1 takes you to one of them; the exercise to all three
- Run checklist items 1 (names), 6 (categories) and 8 (IDs and keys) on *every* file, not only the ones you need
- Write each as an erratum (a correction note): the codebook says · the data say · how you found it

- Where the codebook and the data disagree, the data are what you clean.

The same holds for SCB's documentation on MONA: variable names, code lists and special codes (such as $-9$) change between deliveries.